

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0510

Slot A

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage:

40%

QUESTIONS:3

Total Marks 30

Duration :60 Minutes

Annexure A

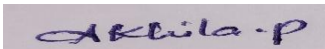
Experiments

Code of Conduct:

I AKHILA POCHIMIREDDY(192425225) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	
---------------	----	---	--	--	----------------------------------	--

1. Preprocessing Impact Analysis

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

Aim

To study the impact of preprocessing techniques such as **noise removal** and **normalization** on image quality and feature detection.

Algorithm

1. Read the input image.
2. Convert the image into grayscale.
3. Add noise (optional for testing).
4. Apply Gaussian Blur for noise removal.
5. Normalize the image intensity.
6. Detect features using Canny Edge Detector.
7. Display original, noisy, preprocessed, normalized, and edge images.
8. Compare the improvement in feature detection.

Python Code

```
import cv2

import matplotlib.pyplot as plt

img = cv2.imread("cv-1.png")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray, (5,5), 0)

norm = cv2.normalize(blur, None, 0, 255, cv2.NORM_MINMAX)

edges = cv2.Canny(norm, 100, 200)

titles = ["Original", "Gray", "Blur", "Normalized", "Edges"]
```

```

images = [cv2.cvtColor(img, cv2.COLOR_BGR2RGB),
          gray,
          blur,
          norm,
          edges]

plt.figure(figsize=(12,6))

for i in range(5):

    plt.subplot(2,3,i+1)

    plt.imshow(images[i], cmap='gray')

    plt.title(titles[i])

    plt.axis("off")

plt.tight_layout()

plt.show()

```

Output



Result

The preprocessing techniques effectively reduced image noise and enhanced image quality. Feature detection after preprocessing produced clearer and more reliable edges than the original image.

2. Image Representation Study

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

Aim

To convert an image into **grayscale** and **binary** representations and analyze their effect on feature extraction.

Algorithm

1. Read the image.
2. Convert RGB image into grayscale.
3. Convert grayscale image into binary using thresholding.
4. Apply edge detection.
5. Compare grayscale and binary representations.

Python Code

```
import cv2

import matplotlib.pyplot as plt

img = cv2.imread("cv-4.png")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

edges = cv2.Canny(gray,100,200)

titles = ["Original","Grayscale","Binary","Edges"]

images = [cv2.cvtColor(img,cv2.COLOR_BGR2RGB),

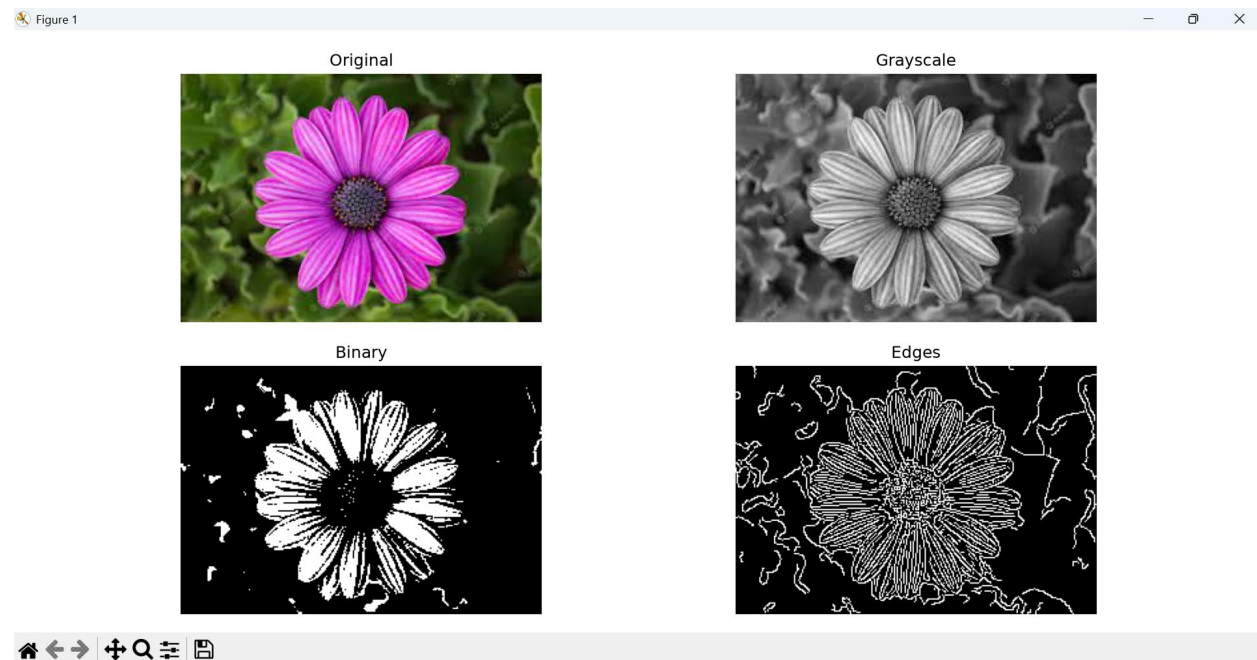
          gray,

          binary,

          edges]
```

```
plt.figure(figsize=(10,5))
for i in range(4):
    plt.subplot(2,2,i+1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis("off")
plt.tight_layout()
plt.show()
```

Output



Result

Grayscale representation preserved image intensity while reducing computational complexity. Binary representation simplified the image for segmentation but lost intensity information. Grayscale images produced better edge detection and feature extraction.

3. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and

analyze the comparative effectiveness of each method for different image conditions and applications.

Aim

To compare **Sobel** and **Canny** edge detection techniques using the same input image.

Algorithm

1. Read the image.
2. Convert to grayscale.
3. Apply Sobel operator in X and Y directions.
4. Compute Sobel magnitude.
5. Apply Canny edge detector.
6. Compare both outputs.

Python Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread("cv-5.png")

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

sobelx = cv2.Sobel(gray, cv2.CV_64F,1,0,ksize=3)

sobely = cv2.Sobel(gray, cv2.CV_64F,0,1,ksize=3)

sobel = cv2.magnitude(sobelx,sobely)

canny = cv2.Canny(gray,100,200)

titles=["Original","Sobel","Canny"]

images=[cv2.cvtColor(img,cv2.COLOR_BGR2RGB),

        sobel,

        canny]

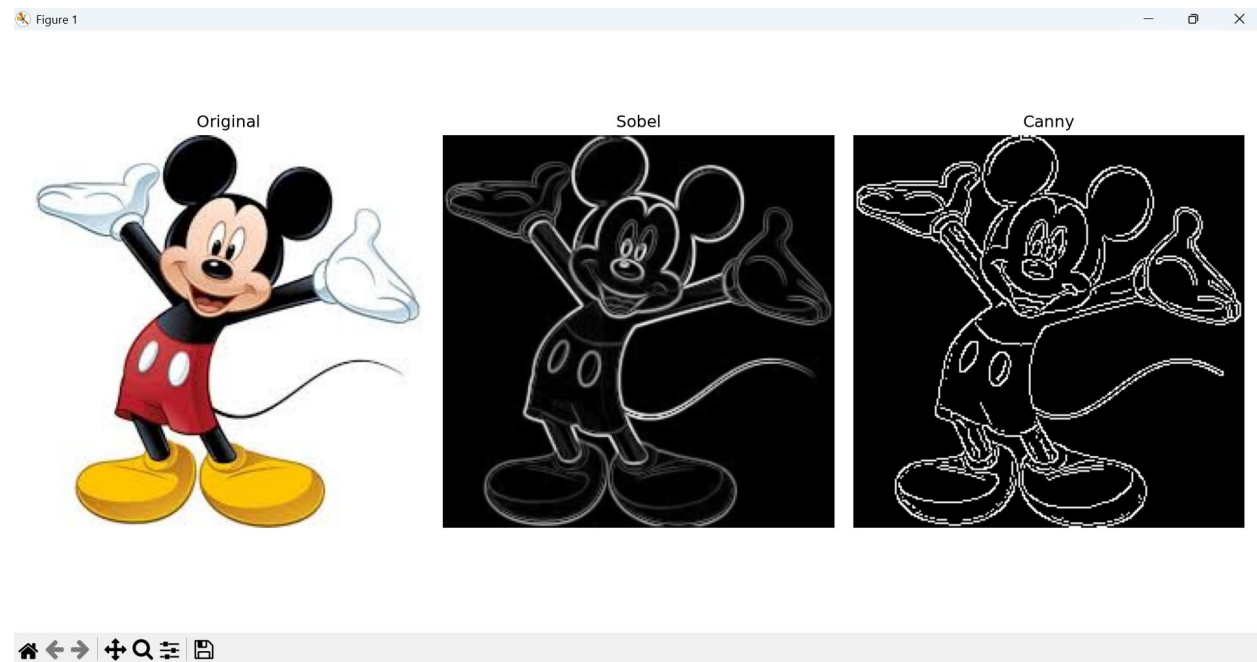
plt.figure(figsize=(10,4))

for i in range(3):

    plt.subplot(1,3,i+1)
```

```
plt.imshow(images[i], cmap='gray')  
  
plt.title(titles[i])  
  
plt.axis("off")  
  
plt.tight_layout()  
  
plt.show()
```

Output



Result

The Sobel operator detected image gradients effectively but produced thicker and noisier edges. The Canny algorithm generated thinner, sharper, and more accurate edges with better noise suppression, making it more suitable for feature detection in computer vision applications.